

consumption. If you have an app with a managed heap larger than 100MB then focus on reducing the garbage collection time only. Here's how you can help the .NET garbage collector achieve better performance.

Reduce memory consumption

Release references

A reference to an object in your app prevents that object, and all of the objects it references, from being collected. The .NET compiler does a good job of detecting when a variable is no longer in use so objects held onto by that variable will be eligible for collection. But in some cases it may not be obvious that some objects have a reference to other objects because part of the object graph might be owned by libraries your app uses. To learn about the tools and techniques to find out which objects survive a garbage collection, see [Garbage collection and performance](#).

Induce a garbage collection if it's useful

Induce a garbage collection only after you have measured your app's performance and have determined that inducing a collection will improve its performance.

You can induce a garbage collection of a generation by calling [GC.Collect\(n\)](#), where n is the generation you want to collect (0, 1, or 2).

Note We recommend that you don't force a garbage collection in your app because the garbage collector uses many heuristics to determine the best time to perform a collection, and forcing a collection is in many cases an unnecessary use of the CPU. But if you know that you have a large number of objects in your app that are no longer used and you want to return this memory to the system, then it may be appropriate to force a garbage collection. For example, you can induce a collection at the end of a loading sequence in a game to free up memory before gameplay starts. To avoid inadvertently inducing too many garbage collections, you can set the [GCCollectionMode](#) to **Optimized**. This instructs the garbage collector to start a collection only if it determines that the collection would be productive enough to be justified.

Reduce garbage collection time

This section applies if you've analyzed your app and observed large garbage collection times. Garbage collection-related pause times include: the time it takes to run a single garbage collection pass; and the total time your app spends doing garbage collections.

The amount of time it takes to do a collection depends on how much live data the collector has to analyze. Generation 0 and generation 1 are bounded in size, but generation 2 continues to grow as more long-lived objects are active in your app. This means that the collection times for generation 0 and generation 1 are bounded, while generation 2 collections can take longer. How often garbage collections run depends mostly on how much memory you allocate, because a garbage collection frees up memory to satisfy allocation requests.

The garbage collector occasionally pauses your app to perform work, but doesn't necessarily pause your app the entire time it is doing a collection. Pause times are usually not user-perceivable in your app, especially for generation 0 and generation 1 collections. The [Background garbage collection](#) feature of the .NET garbage collector allows Generation 2 collections to be performed concurrently while your app is running and will only pause your app for short periods of time. But it is not always possible to do a Generation 2 collection as a background collection. In that case, the pause can be user-perceivable if you have a large enough heap (more than 100MB).

Frequent garbage collections can contribute to increased CPU (and therefore power) consumption, longer loading times, or decreased frame rates in your application. Below are some techniques you can use to reduce garbage collection time and collection-related pauses in your managed UWP app.

Reduce memory allocations

If you don't allocate any objects then the garbage collector doesn't run unless there is a low memory condition in the system. Reducing the amount of memory you allocate directly translates to less frequent garbage collections.

If in some sections of your app pauses are completely undesirable, then you can pre-allocate the necessary objects beforehand during a less performance-critical time. For example, a game might allocate all of the objects needed for gameplay during the loading screen of a level and not make any allocations during gameplay. This avoids pauses while the user is playing the game and can result in a higher and more consistent frame rate.

Reduce generation 2 collections by avoiding objects with a medium-length lifetime

Generational garbage collections perform best when you have really short-lived and/or really long-lived objects in your app. Short lived objects are collected in the cheaper generation 0 and generation 1 collections, and objects that are long-lived get promoted

to generation 2, which is collected infrequently. Long-lived objects are those that are in use for the entire duration of your app, or during a significant period of your app, such as during a specific page or game level.

If you frequently create objects that have a temporary lifetime but live long enough to be promoted to generation 2, then more of the expensive generation 2 collections happen. You may be able to reduce generation 2 collections by recycling existing objects or releasing objects more quickly.

A common example of objects with medium-term lifetime is objects that are used for displaying items in a list that a user scrolls through. If objects are created when items in the list are scrolled into view, and are no longer referenced as items in the list are scrolled out of view, then your app typically has a large number of generation 2 collections. In situations like this you can pre-allocate and reuse a set of objects for the data that is actively shown to the user, and use short-lived objects to load info as items in the list come into view.

Reduce generation 2 collections by avoiding large-sized objects with short lifetimes

Any object that is 85KB or larger is allocated on the large object heap (LOH) and gets collected as part of generation 2. If you have temporary variables, such as buffers, that are greater than 85KB, then a generation 2 collection cleans them up. Limiting temporary variables to less than 85KB reduces the number of generation 2 collections in your app. One common technique is to create a buffer pool and reuse objects from the pool to avoid large temporary allocations.

Avoid reference-rich objects

The garbage collector determines which objects are live by following references between objects, starting from roots in your app. For more info, see [What happens during a garbage collection](#). If an object contains many references, then there is more work for the garbage collector to do. A common technique (especially with large objects) is to convert reference rich objects into objects with no references (for example, instead of storing a reference, store an index). Of course this technique works only when it is logically possible to do so.

Replacing object references with indexes can be a disruptive and complicated change to your app and is most effective for large objects with a large number of references. Do this only if you are noticing large garbage collection times in your app related to reference-heavy objects.

Keep the UI thread responsive

Article • 10/20/2022

Users expect an app to remain responsive while it does computation, regardless of the type of machine. This means different things for different apps. For some, this translates to providing more realistic physics, loading data from disk or the web faster, quickly presenting complex scenes and navigating between pages, finding directions in a snap, or rapidly processing data. Regardless of the type of computation, users want their app to act on their input and eliminate instances where it appears unresponsive while it "thinks".

Your app is event-driven, which means that your code performs work in response to an event and then it sits idle until the next. Platform code for UI (layout, input, raising events, etc.) and your app's code for UI all are executed on the same UI thread. Only one instruction can execute on that thread at a time so if your app code takes too long to process an event then the framework can't run layout or raise new events representing user interaction. The responsiveness of your app is related to the availability of the UI thread to process work.

You need to use the UI thread to make almost all changes to the UI thread, including creating UI types and accessing their members. You can't update the UI from a background thread but you can post a message to it with [CoreDispatcher.RunAsync](#) to cause code to be run there.

Note The one exception is that there's a separate render thread that can apply UI changes that won't affect how input is handled or the basic layout. For example many animations and transitions that don't affect layout can run on this render thread.

Delay element instantiation

Some of the slowest stages in an app can include startup, and switching views. Don't do more work than necessary to bring up the UI that the user sees initially. For example, don't create the UI for progressively-disclosed UI and the contents of popups.

- Use [x:Load attribute](#) or [x:DeferLoadStrategy](#) to delay-instantiate elements.
- Programmatically insert elements into the tree on-demand.

[CoreDispatcher.RunIdleAsync](#) queues work for the UI thread to process when it's not busy.

Use asynchronous APIs

To help keep your app responsive, the platform provides asynchronous versions of many of its APIs. An asynchronous API ensures that your active execution thread never blocks for a significant amount of time. When you call an API from the UI thread, use the asynchronous version if it's available. For more info about programming with **async** patterns, see [Asynchronous programming](#) or [Call asynchronous APIs in C# or Visual Basic](#).

Offload work to background threads

Write event handlers to return quickly. In cases where a non-trivial amount of work needs to be performed, schedule it on a background thread and return.

You can schedule work asynchronously by using the **await** operator in C#, the **Await** operator in Visual Basic, or delegates in C++. But this doesn't guarantee that the work you schedule will run on a background thread. Many of the Universal Windows Platform (UWP) APIs schedule work in the background thread for you, but if you call your app code by using only **await** or a delegate, you run that delegate or method on the UI thread. You have to explicitly say when you want to run your app code on a background thread. In C# and Visual Basic you can accomplish this by passing code to [Task.Run](#).

Remember that UI elements may only be accessed from the UI thread. Use the UI thread to access UI elements before launching the background work and/or use [CoreDispatcher.RunAsync](#) or [CoreDispatcher.RunIdleAsync](#) on the background thread.

An example of work that can be performed on a background thread is the calculating of computer AI in a game. The code that calculates the computer's next move can take a lot of time to execute.

C#

```
public class AsyncExample
{
    private async void NextMove_Click(object sender, RoutedEventArgs e)
    {
        // The await causes the handler to return immediately.
        await System.Threading.Tasks.Task.Run(() => ComputeNextMove());
        // Now update the UI with the results.
        // ...
    }

    private async System.Threading.Tasks.Task ComputeNextMove()
    {
        // Perform background work here.
    }
}
```

```
        // Don't directly access UI elements from this method.  
    }  
}
```

VB

```
Public Class AsyncExample  
    ' ...  
    Private Async Sub NextMove_Click(ByVal sender As Object, ByVal e As  
RoutedEventArgs)  
        Await Task.Run(Function() ComputeNextMove())  
        ' update the UI with results  
    End Sub  
  
    Private Async Function ComputeNextMove() As Task  
        ' ...  
    End Function  
    ' ...  
End Class
```

In this example, the `NextMove_Click` handler returns at the **await** in order to keep the UI thread responsive. But execution picks up in that handler again after `ComputeNextMove` (which executes on a background thread) completes. The remaining code in the handler updates the UI with the results.

Note There's also a [ThreadPool](#) and [ThreadPoolTimer](#) API for the UWP, which can be used for similar scenarios. For more info, see [Threading and async programming](#).

Related topics

- [Custom user interactions](#)

Optimize your XAML markup

Article • 10/20/2022

Parsing XAML markup to construct objects in memory is time-consuming for a complex UI. Here are some things you can do to improve the parse and load time of your XAML markup and the memory efficiency of your app.

At app startup, limit the XAML markup that is loaded to only what you need for your initial UI. Examine the markup in your initial page (including page resources) and confirm that you aren't loading extra elements that aren't needed right away. These elements can come from a variety of sources, such as resource dictionaries, elements that are initially collapsed, and elements drawn over other elements.

Optimizing your XAML for efficiency requires making trade-offs; there's not always a single solution for every situation. Here, we look at some common issues and provide guidelines you can use to make the right trade-offs for your app.

Minimize element count

Although the XAML platform is capable of displaying large numbers of elements, you can make your app lay out and render faster by using the fewest number of elements to achieve the visuals you want.

The choices you make in how you lay out your UI controls will affect the number of UI elements that are created when your app starts. For more detailed information about optimizing layout, see [Optimize your XAML layout](#).

Element count is extremely important in data templates because each element is created again for each data item. For info about reducing element count in a list or grid, see *Element reduction per item* in the [ListView and GridView UI optimization](#) article.

Here, we look at some other ways you can reduce the number of elements your app has to load at startup.

Defer item creation

If your XAML markup contains elements that you don't show right away, you can defer loading those elements until they are shown. For example, you can delay the creation of non-visible content such as a secondary tab in a tab-like UI. Or, you might show items in a grid view by default, but provide an option for the user to view the data in a list instead. You can delay loading the list until it's needed.

Use the [x:Load attribute](#) instead of the [Visibility](#) property to control when an element is shown. When an element's visibility is set to **Collapsed**, then it is skipped during the render pass, but you still pay the object instance costs in memory. When you use [x:Load](#) instead, the framework does not create the object instance until it is needed, so the memory costs are even lower. The drawback is you pay a small memory overhead (approx 600 bytes) when the UI is not loaded.

❗ Note

You can delay loading elements using the [x:Load](#) or [x:DeferLoadStrategy](#) attribute. The [x:Load](#) attribute is available starting in Windows 10 Creators Update (version 1703, SDK build 15063). The min version targeted by your Visual Studio project must be *Windows 10 Creators Update (10.0, Build 15063)* in order to use [x:Load](#). To target earlier versions, use [x:DeferLoadStrategy](#).

The following examples show the difference in element count and memory use when different techniques are used to hide UI elements. A [ListView](#) and a [GridView](#) containing identical items are placed in a page's root [Grid](#). The [ListView](#) is not visible, but the [GridView](#) is shown. The XAML in each of these examples produces the same UI on the screen. We use Visual Studio's [tools for profiling and performance](#) to check the element count and memory use.

Option 1 - Inefficient

Here, the [ListView](#) is loaded, but is not visible because its [Width](#) is 0. The [ListView](#) and each of its child elements is created in the visual tree and loaded into memory.

XAML

```
<!-- NOTE: EXAMPLE OF INEFFICIENT CODE; DO NOT COPY-PASTE.-->
<Grid Background="{ThemeResource ApplicationPageBackgroundThemeBrush}">

    <ListView x:Name="List1" Width="0">
        <ListViewItem>Item 1</ListViewItem>
        <ListViewItem>Item 2</ListViewItem>
        <ListViewItem>Item 3</ListViewItem>
        <ListViewItem>Item 4</ListViewItem>
        <ListViewItem>Item 5</ListViewItem>
        <ListViewItem>Item 6</ListViewItem>
        <ListViewItem>Item 7</ListViewItem>
        <ListViewItem>Item 8</ListViewItem>
        <ListViewItem>Item 9</ListViewItem>
        <ListViewItem>Item 10</ListViewItem>
    </ListView>
```

```

<GridView x:Name="Grid1">
    <GridViewItem>Item 1</GridViewItem>
    <GridViewItem>Item 2</GridViewItem>
    <GridViewItem>Item 3</GridViewItem>
    <GridViewItem>Item 4</GridViewItem>
    <GridViewItem>Item 5</GridViewItem>
    <GridViewItem>Item 6</GridViewItem>
    <GridViewItem>Item 7</GridViewItem>
    <GridViewItem>Item 8</GridViewItem>
    <GridViewItem>Item 9</GridViewItem>
    <GridViewItem>Item 10</GridViewItem>
</GridView>
</Grid>

```

Live visual tree with the ListView loaded. Total element count for the page is 89.

```

  1  <> [MainPage] (89)
    1  [Grid] (88)
        1  List1 [ListView] (43)
        1  Grid1 [GridView] (43)

```

ListView and its children are loaded into memory.

Managed Memory (TestApp1.exe)

Object Type	Count	Size (Bytes)	Inclusive Size (Bytes)
TestApp1.MainPage	1	24	48
Windows.UI.Xaml.Controls.ListView	1	12	12
Windows.UI.Xaml.Controls.GridView	1	12	12

Option 2 - Better

Here, the ListView's Visibility is set to collapsed (the other XAML is identical to the original). The ListView is created in the visual tree, but its child elements are not. However, they are loaded into memory, so memory use is identical to the previous example.

XAML

```

<ListView x:Name="List1" Visibility="Collapsed">

```

Live visual tree with the ListView collapsed. Total element count for the page is 46.

▲ <> [MainPage] (46)
 ▲ [Grid] (45)
 List1 [ListView]
 ▶ Grid1 [GridView] (43)

ListView and its children are loaded into memory.

Managed Memory (TestApp1.exe)

Object Type	Count ▼	Size (Bytes)	Inclusive Size (Bytes)
TestApp1.MainPage	1	24	48
Windows.UI.Xaml.Controls.ListView	1	12	12
Windows.UI.Xaml.Controls.GridView	1	12	12

Option 3 - Most Efficient

Here, the ListView has the x:Load attribute set to **False** (the other XAML is identical to the original). The ListView is not created in the visual tree or loaded into memory at start up.

XAML

```
<ListView x:Name="List1" Visibility="Collapsed" x:Load="False">
```

Live visual tree with the ListView not loaded. Total element count for the page is 45.

▲ <> [MainPage] (45)
 ▲ [Grid] (44)
 ▶ Grid1 [GridView] (43)

ListView and its children are not loaded into memory.

Managed Memory (TestApp1.exe)

Object Type	Count ▼	Size (Bytes)	Inclusive Size (Bytes)
TestApp1.TestApp1_XamlTypeInfo.Activat...	1	32	32
TestApp1.MainPage	1	24	36
Windows.UI.Xaml.Controls.GridView	1	12	12

ⓘ Note

The element counts and memory use in these examples are very small and are shown only to demonstrate the concept. In these examples, the overhead of using `x:Load` is greater than the memory savings, so the app would not benefit. You should use the profiling tools on your app to determine whether or not your app will benefit from deferred loading.

Use layout panel properties

Layout panels have a [Background](#) property so there's no need to put a [Rectangle](#) in front of a Panel just to color it.

Inefficient

XAML

```
<!-- NOTE: EXAMPLE OF INEFFICIENT CODE; DO NOT COPY-PASTE. -->
<Grid>
    <Rectangle Fill="Black"/>
</Grid>
```

Efficient

XAML

```
<Grid Background="Black"/>
```

Layout panels also have built-in border properties, so you don't need to put a [Border](#) element around a layout panel. See [Optimize your XAML layout](#) for more info and examples.

Use images in place of vector-based elements

If you reuse the same vector-based element enough times, it becomes more efficient to use an [Image](#) element instead. Vector-based elements can be more expensive because the CPU must create each individual element separately. The image file needs to be decoded only once.

Optimize resources and resource dictionaries

You typically use [resource dictionaries](#) to store, at a somewhat global level, resources that you want to reference in multiple places in your app. For example, styles, brushes,

templates, and so on.

In general, we have optimized [ResourceDictionary](#) to not instantiate resources unless they're asked for. But there are situations you should avoid so that resources aren't instantiated unnecessarily.

Resources with x:Name

Use the [x:Key attribute](#) to reference your resources. Any resource with the [x:Name attribute](#) will not benefit from the platform optimization; instead, it is instantiated as soon as the ResourceDictionary is created. This happens because x:Name tells the platform that your app needs field access to this resource, so the platform needs to create something to create a reference to.

ResourceDictionary in a UserControl

A ResourceDictionary defined inside of a [UserControl](#) carries a penalty. The platform creates a copy of such a ResourceDictionary for every instance of the UserControl. If you have a UserControl that is used a lot, then move the ResourceDictionary out of the UserControl and put it at the page level.

Resource and ResourceDictionary scope

If a page references a user control or a resource defined in a different file, then the framework parses that file, too.

Here, because *InitialPage.xaml* uses one resource from *ExampleResourceDictionary.xaml*, the whole of *ExampleResourceDictionary.xaml* must be parsed at startup.

InitialPage.xaml.

XAML

```
<Page x:Class="ExampleNamespace.InitialPage" ...>
  <Page.Resources>
    <ResourceDictionary>
      <ResourceDictionary.MergedDictionaries>
        <ResourceDictionary
Source="ExampleResourceDictionary.xaml"/>
      </ResourceDictionary.MergedDictionaries>
    </ResourceDictionary>
  </Page.Resources>

  <Grid>
    <TextBox Foreground="{StaticResource TextBrush}"/>
```

```
</Grid>
</Page>
```

ExampleResourceDictionary.xaml.

XAML

```
<ResourceDictionary>
  <SolidColorBrush x:Key="TextBrush" Color="#FF3F42CC"/>

  <!--This ResourceDictionary contains many other resources that
       are used in the app, but are not needed during startup.-->
</ResourceDictionary>
```

If you use a resource on many pages throughout your app, then storing it in *App.xaml* is a good practice, and avoids duplication. But *App.xaml* is parsed at app startup so any resource that is used in only one page (unless that page is the initial page) should be put into the page's local resources. This example shows *App.xaml* containing resources that are used by only one page (that's not the initial page). This needlessly increases app startup time.

App.xaml

XAML

```
<!-- NOTE: EXAMPLE OF INEFFICIENT CODE; DO NOT COPY-PASTE. -->
<Application ...>
  <Application.Resources>
    <SolidColorBrush x:Key="DefaultAppTextBrush" Color="#FF3F42CC"/>
    <SolidColorBrush x:Key="InitialPageTextBrush" Color="#FF3F42CC"/>
    <SolidColorBrush x:Key="SecondPageTextBrush" Color="#FF3F42CC"/>
    <SolidColorBrush x:Key="ThirdPageTextBrush" Color="#FF3F42CC"/>
  </Application.Resources>
</Application>
```

InitialPage.xaml.

XAML

```
<!-- NOTE: EXAMPLE OF INEFFICIENT CODE; DO NOT COPY-PASTE. -->
<Page x:Class="ExampleNamespace.InitialPage" ...>
  <StackPanel>
    <TextBox Foreground="{StaticResource InitialPageTextBrush}"/>
  </StackPanel>
</Page>
```

SecondPage.xaml.

XAML

```
<!-- NOTE: EXAMPLE OF INEFFICIENT CODE; DO NOT COPY-PASTE. -->
<Page x:Class="ExampleNamespace.SecondPage" ...>
    <StackPanel>
        <Button Content="Submit" Foreground="{StaticResource
SecondPageTextBrush}" />
    </StackPanel>
</Page>
```

To make this example more efficient, move `SecondPageTextBrush` into *SecondPage.xaml* and move `ThirdPageTextBrush` into *ThirdPage.xaml*. `InitialPageTextBrush` can remain in *App.xaml* because application resources must be parsed at app startup in any case.

Consolidate multiple brushes that look the same into one resource

The XAML platform tries to cache commonly-used objects so that they can be reused as often as possible. But XAML cannot easily tell if a brush declared in one piece of markup is the same as a brush declared in another. The example here uses `SolidColorBrush` to demonstrate, but the case is more likely and more important with `GradientBrush`. Also check for brushes that use predefined colors; for example, `"Orange"` and `"#FFFFA500"` are the same color.

Inefficient.

XAML

```
<!-- NOTE: EXAMPLE OF INEFFICIENT CODE; DO NOT COPY-PASTE. -->
<Page ... >
    <StackPanel>
        <TextBlock>
            <TextBlock.Foreground>
                <SolidColorBrush Color="#FFFFA500" />
            </TextBlock.Foreground>
        </TextBlock>
        <Button Content="Submit">
            <Button.Foreground>
                <SolidColorBrush Color="#FFFFA500" />
            </Button.Foreground>
        </Button>
    </StackPanel>
</Page>
```

To fix the duplication, define the brush as a resource. If controls in other pages use the same brush, move it to *App.xaml*.

Efficient.

XAML

```
<Page ... >
  <Page.Resources>
    <SolidColorBrush x:Key="BrandBrush" Color="#FFFA500"/>
  </Page.Resources>

  <StackPanel>
    <TextBlock Foreground="{StaticResource BrandBrush}" />
    <Button Content="Submit" Foreground="{StaticResource BrandBrush}" />
  </StackPanel>
</Page>
```

Minimize overdrawing

Overdrawing occurs where more than one object is drawn in the same screen pixels. Note that there is sometimes a trade-off between this guidance and the desire to minimize element count.

Use [DebugSettings.IsOverdrawHeatMapEnabled](#) as a visual diagnostic. You might find objects being drawn that you didn't know were in the scene.

Transparent or hidden elements

If an element isn't visible because it's transparent or hidden behind other elements, and it's not contributing to layout, then delete it. If the element is not visible in the initial visual state but it is visible in other visual states, then use `x:Load` to control its state or set [Visibility](#) to **Collapsed** on the element itself and change the value to **Visible** in the appropriate states. There will be exceptions to this heuristic: in general, the value a property has in the majority of visual states is best set locally on the element.

Composite elements

Use a composite element instead of layering multiple elements to create an effect. In this example, the result is a two-toned shape where the top half is black (from the background of the [Grid](#)) and the bottom half is gray (from the semi-transparent white [Rectangle](#) alpha-blended over the black background of the **Grid**). Here, 150% of the pixels necessary to achieve the result are being filled.

Inefficient.

XAML

```
<!-- NOTE: EXAMPLE OF INEFFICIENT CODE; DO NOT COPY-PASTE. -->
<Grid Background="Black">
    <Grid.RowDefinitions>
        <RowDefinition Height="*" />
        <RowDefinition Height="*" />
    </Grid.RowDefinitions>
    <Rectangle Grid.Row="1" Fill="White" Opacity=".5" />
</Grid>
```

Efficient.

XAML

```
<Grid>
    <Grid.RowDefinitions>
        <RowDefinition Height="*" />
        <RowDefinition Height="*" />
    </Grid.RowDefinitions>
    <Rectangle Fill="Black" />
    <Rectangle Grid.Row="1" Fill="#FF7F7F7F" />
</Grid>
```

Layout panels

A layout panel can have two purposes: to color an area, and to lay out child elements. If an element further back in z-order is already coloring an area then a layout panel in front does not need to paint that area: instead it can just focus on laying out its children. Here's an example.

Inefficient.

XAML

```
<!-- NOTE: EXAMPLE OF INEFFICIENT CODE; DO NOT COPY-PASTE. -->
<GridView Background="Blue">
    <GridView.ItemTemplate>
        <DataTemplate>
            <Grid Background="Blue" />
        </DataTemplate>
    </GridView.ItemTemplate>
</GridView>
```

Efficient.

XAML

```
<GridView Background="Blue">
    <GridView.ItemTemplate>
        <DataTemplate>
            <Grid/>
        </DataTemplate>
    </GridView.ItemTemplate>
</GridView>
```

If the [Grid](#) has to be hit-testable then set a background value of transparent on it.

Borders

Use a [Border](#) element to draw a border around an object. In this example, a [Grid](#) is used as a makeshift border around a [TextBox](#). But all the pixels in the center cell are overdrawn.

Inefficient.

XAML

```
<!-- NOTE: EXAMPLE OF INEFFICIENT CODE; DO NOT COPY-PASTE. -->
<Grid Background="Blue" Width="300" Height="45">
    <Grid.RowDefinitions>
        <RowDefinition Height="5"/>
        <RowDefinition/>
        <RowDefinition Height="5"/>
    </Grid.RowDefinitions>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="5"/>
        <ColumnDefinition/>
        <ColumnDefinition Width="5"/>
    </Grid.ColumnDefinitions>
    <TextBox Grid.Row="1" Grid.Column="1"></TextBox>
</Grid>
```

Efficient.

XAML

```
<Border BorderBrush="Blue" BorderThickness="5" Width="300" Height="45">
    <TextBox/>
</Border>
```

Margins

Be aware of margins. Two neighboring elements will overlap (possibly accidentally) if negative margins extend into another's render bounds and cause overdrawing.

Cache static content

Another source of overdrawing is a shape made from many overlapping elements. If you set `CacheMode` to `BitmapCache` on the `UIElement` that contains the composite shape then the platform renders the element to a bitmap once and then uses that bitmap each frame instead of overdrawing.

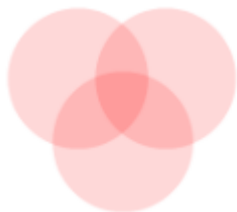
Inefficient.

XAML

```
<Canvas Background="White">
  <Ellipse Height="40" Width="40" Fill="Blue"/>
  <Ellipse Canvas.Left="21" Height="40" Width="40" Fill="Blue"/>
  <Ellipse Canvas.Top="13" Canvas.Left="10" Height="40" Width="40"
Fill="Blue"/>
</Canvas>
```



The image above is the result, but here's a map of the overdrawn regions. Darker red indicates higher amounts of overdraw.



Efficient.

XAML

```
<Canvas Background="White" CacheMode="BitmapCache">
  <Ellipse Height="40" Width="40" Fill="Blue"/>
  <Ellipse Canvas.Left="21" Height="40" Width="40" Fill="Blue"/>
  <Ellipse Canvas.Top="13" Canvas.Left="10" Height="40" Width="40"
Fill="Blue"/>
</Canvas>
```

Note the use of [CacheMode](#). Don't use this technique if any of the sub-shapes animate because the bitmap cache will likely need to be regenerated every frame, defeating the purpose.

Use XBF2

XBF2 is a binary representation of XAML markup that avoids all text-parsing costs at runtime. It also optimizes your binary for load and tree creation, and allows "fast-path" for XAML types to improve heap and object creation costs, for example VSM, ResourceDictionary, Styles, and so on. It is completely memory-mapped so there is no heap footprint for loading and reading a XAML Page. In addition, it reduces the disk footprint of stored XAML pages in an appx. XBF2 is a more compact representation and it can reduce disk footprint of comparative XAML/XBF1 files by up to 50%. For example, the built-in Photos app saw around a 60% reduction after conversion to XBF2 dropping from around ~1mb of XBF1 assets to ~400kb of XBF2 assets. We have also seen apps benefit anywhere from 15 to 20% in CPU and 10 to 15% in Win32 heap.

XAML built-in controls and dictionaries that the framework provides are already fully XBF2-enabled. For your own app, ensure that your project file declares TargetPlatformVersion 8.2 or later.

To check whether you have XBF2, open your app in a binary editor; the 12th and 13th bytes are 00 02 if you have XBF2.

Related articles

- [Best practices for your app's startup performance](#)
- [Optimize your XAML layout](#)
- [ListView and GridView UI optimization](#)
- [Tools for profiling and performance](#)

Optimize your XAML layout

Article • 10/20/2022

Important APIs

- [Panel](#)

Layout is the process of defining the visual structure for your UI. The primary mechanism for describing layout in XAML is through panels, which are container objects that enable you to position and arrange the UI elements within them. Layout can be an expensive part of a XAML app—both in CPU usage and memory overhead. Here are some simple steps you can take to improve the layout performance of your XAML app.

Reduce layout structure

The biggest gain in layout performance comes from simplifying the hierarchical structure of the tree of UI elements. Panels exist in the visual tree, but they are structural elements, not *pixel producing elements* like a [Button](#) or a [Rectangle](#). Simplifying the tree by reducing the number of non-pixel-producing elements typically provides a significant performance increase.

Many UIs are implemented by nesting panels which results in deep, complex trees of panels and elements. It is convenient to nest panels, but in many cases the same UI can be achieved with a more complex single panel. Using a single panel provides better performance.

When to reduce layout structure

Reducing layout structure in a trivial way—for example, reducing one nested panel from your top-level page—does not have a noticeable effect.

The largest performance gains come from reducing layout structure that's repeated in the UI, like in a [ListView](#) or [GridView](#). These [ItemsControl](#) elements use a [DataTemplate](#), which defines a subtree of UI elements that is instantiated many times. When the same subtree is being duplicated many times in your app, any improvements to the performance of that subtree has a multiplicative effect on the overall performance of your app.

Examples

Consider the following UI.

Options:

☐ Power User ☐ Admin

Basic information:

Name:

Email:

Password:

Save

These examples shows 3 ways of implementing the same UI. Each implementation choice results in nearly identical pixels on the screen, but differs substantially in the implementation details.

Option1: Nested **StackPanel** elements

Although this is the simplest model, it uses 5 panel elements and results in significant overhead.

XML

```
<StackPanel>
  <TextBlock Text="Options:" />
  <StackPanel Orientation="Horizontal">
    <CheckBox Content="Power User" />
    <CheckBox Content="Admin" Margin="20,0,0,0" />
  </StackPanel>
  <TextBlock Text="Basic information:" />
  <StackPanel Orientation="Horizontal">
    <TextBlock Text="Name:" Width="75" />
    <TextBox Width="200" />
  </StackPanel>
  <StackPanel Orientation="Horizontal">
    <TextBlock Text="Email:" Width="75" />
    <TextBox Width="200" />
  </StackPanel>
  <StackPanel Orientation="Horizontal">
    <TextBlock Text="Password:" Width="75" />
    <TextBox Width="200" />
  </StackPanel>
  <Button Content="Save" />
</StackPanel>
```

Option 2: A single **Grid**

The **Grid** adds some complexity, but uses only a single panel element.

XML

```
<Grid>
  <Grid.RowDefinitions>
    <RowDefinition Height="Auto" />
    <RowDefinition Height="Auto" />
    <RowDefinition Height="Auto" />
    <RowDefinition Height="Auto" />
    <RowDefinition Height="Auto" />
    <RowDefinition Height="Auto" />
    <RowDefinition Height="Auto" />
  </Grid.RowDefinitions>
  <Grid.ColumnDefinitions>
    <ColumnDefinition Width="Auto" />
    <ColumnDefinition Width="Auto" />
  </Grid.ColumnDefinitions>
  <TextBlock Text="Options:" Grid.ColumnSpan="2" />
  <CheckBox Content="Power User" Grid.Row="1" Grid.ColumnSpan="2" />
  <CheckBox Content="Admin" Margin="150,0,0,0" Grid.Row="1"
Grid.ColumnSpan="2" />
  <TextBlock Text="Basic information:" Grid.Row="2" Grid.ColumnSpan="2" />
  <TextBlock Text="Name:" Width="75" Grid.Row="3" />
  <TextBox Width="200" Grid.Row="3" Grid.Column="1" />
  <TextBlock Text="Email:" Width="75" Grid.Row="4" />
  <TextBox Width="200" Grid.Row="4" Grid.Column="1" />
  <TextBlock Text="Password:" Width="75" Grid.Row="5" />
  <TextBox Width="200" Grid.Row="5" Grid.Column="1" />
  <Button Content="Save" Grid.Row="6" />
</Grid>
```

Option 3: A single **RelativePanel**:

This single panel is also a bit more complex than using nested panels, but may be easier to understand and maintain than a **Grid**.

XML

```
<RelativePanel>
  <TextBlock Text="Options:" x:Name="Options" />
  <CheckBox Content="Power User" x:Name="PowerUser"
RelativePanel.Below="Options" />
  <CheckBox Content="Admin" Margin="20,0,0,0"
RelativePanel.RightOf="PowerUser" RelativePanel.Below="Options"
/>
  <TextBlock Text="Basic information:" x:Name="BasicInformation"
RelativePanel.Below="PowerUser" />
  <TextBlock Text="Name:" RelativePanel.AlignVerticalCenterWith="NameBox" />
  <TextBox Width="200" Margin="75,0,0,0" x:Name="NameBox"
RelativePanel.Below="BasicInformation" />
  <TextBlock Text="Email:" RelativePanel.AlignVerticalCenterWith="EmailBox"
/>
  <TextBox Width="200" Margin="75,0,0,0" x:Name="EmailBox"
```

```

        RelativePanel.Below="NameBox" />
        <TextBlock Text="Password:"
RelativePanel.AlignVerticalCenterWith="PasswordBox" />
        <TextBox Width="200" Margin="75,0,0,0" x:Name="PasswordBox"
                RelativePanel.Below="EmailBox" />
        <Button Content="Save" RelativePanel.Below="PasswordBox" />
</RelativePanel>

```

As these examples show, there are many ways of achieving the same UI. You should choose by carefully considering all the tradeoffs, including performance, readability, and maintainability.

Use single-cell grids for overlapping UI

A common UI requirement is to have a layout where elements overlap each other. Typically padding, margins, alignments, and transforms are used to position the elements this way. The XAML [Grid](#) control is optimized to improve layout performance for elements that overlap.

Important To see the improvement, use a single-cell [Grid](#). Do not define [RowDefinitions](#) or [ColumnDefinitions](#).

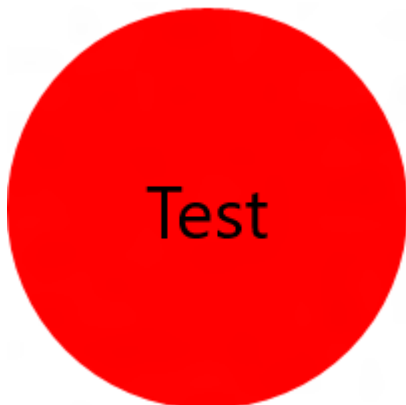
Examples

XML

```

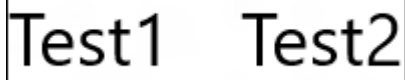
<Grid>
    <Ellipse Fill="Red" Width="200" Height="200" />
    <TextBlock Text="Test"
                HorizontalAlignment="Center"
                VerticalAlignment="Center" />
</Grid>

```



XML

```
<Grid Width="200" BorderBrush="Black" BorderThickness="1">
    <TextBlock Text="Test1" HorizontalAlignment="Left" />
    <TextBlock Text="Test2" HorizontalAlignment="Right" />
</Grid>
```



Use a panel's built-in border properties

[Grid](#), [StackPanel](#), [RelativePanel](#), and [ContentPresenter](#) controls have built-in border properties that let you draw a border around them without adding an additional [Border](#) element to your XAML. The new properties that support the built-in border are: **BorderBrush**, **BorderThickness**, **CornerRadius**, and **Padding**. Each of these is a [DependencyProperty](#), so you can use them with bindings and animations. They're designed to be a full replacement for a separate **Border** element.

If your UI has [Border](#) elements around these panels, use the built-in border instead, which saves an extra element in the layout structure of your app. As mentioned previously, this can be a significant savings, especially in the case of repeated UI.

Examples

XML

```
<RelativePanel BorderBrush="Red" BorderThickness="2" CornerRadius="10"
    Padding="12">
    <TextBox x:Name="textBox1" RelativePanel.AlignLeftWithPanel="True"/>
    <Button Content="Submit" RelativePanel.Below="textBox1"/>
</RelativePanel>
```

Use SizeChanged events to respond to layout changes

The [FrameworkElement](#) class exposes two similar events for responding to layout changes: [LayoutUpdated](#) and [SizeChanged](#). You might be using one of these events to receive notification when an element is resized during layout. The semantics of the two events are different, and there are important performance considerations in choosing between them.

For good performance, **SizeChanged** is almost always the right choice. **SizeChanged** has intuitive semantics. It is raised during layout when the size of the **FrameworkElement** has been updated.

LayoutUpdated is also raised during layout, but it has global semantics—it is raised on every element whenever any element is updated. It is typical to only do local processing in the event handler, in which case the code is run more often than needed. Use **LayoutUpdated** only if you need to know when an element is repositioned without changing size (which is uncommon).

Choosing between panels

Performance is typically not a consideration when choosing between individual panels. That choice is typically made by considering which panel provides the layout behavior that is closest to the UI you're implementing. For example, if you're choosing between **Grid**, **StackPanel**, and **RelativePanel**, you should choose the panel that provides the closest mapping to your mental model of the implementation.

Every XAML panel is optimized for good performance, and all the panels provide similar performance for similar UI.

MVVM and language performance tips

Article • 10/20/2022

This topic discusses some performance considerations related to your choice of software design patterns, and programming language.

The Model-View-ViewModel (MVVM) pattern

The Model-View-ViewModel (MVVM) pattern is common in a lot of XAML apps. (MVVM is very similar to Fowler's description of the Model-View-Presenter pattern, but it is tailored to XAML). The issue with the MVVM pattern is that it can inadvertently lead to apps that have too many layers and too many allocations. The motivations for MVVM are these.

- **Separation of concerns.** It's always helpful to divide a problem into smaller pieces, and a pattern like MVVM or MVC is a way to divide an app (or even a single control) into smaller pieces: the actual view, a logical model of the view (view-model), and the view-independent app logic (the model). In particular, it's a popular workflow to have designers own the view using one tool, developers own the model using another tool, and design integrators own the view-model using both tools.
- **Unit testing.** You can unit test the view-model (and consequently the model) independent of the view, thereby not relying on creating windows, driving input, and so on. By keeping the view small, you can test a large portion of your app without ever having to create a window.
- **Agility to user experience changes.** The view tends to see the most frequent changes, and the most late changes, as the user experience is tweaked based on end-user feedback. By keeping the view separate, these changes can be accommodated more quickly and with less churn to the app.

There are multiple concrete definitions of the MVVM pattern, and 3rd party frameworks that help implement it. But strict adherence to any variation of the pattern can lead to apps with a lot more overhead than can be justified.

- XAML data binding (the {Binding} markup extension) was designed in part to enable model/view patterns. But {Binding} brings with it non-trivial working set and CPU overhead. Creating a {Binding} causes a series of allocations, and updating a binding target can cause reflection and boxing. These problems are being addressed with the {x:Bind} markup extension, which compiles the bindings at build time. **Recommendation:** use {x:Bind}.

- It's popular in MVVM to connect Button.Click to the view-model using an ICommand, such as the common DelegateCommand or RelayCommand helpers. Those commands are extra allocations, though, including the CanExecuteChanged event listener, adding to the working set, and adding to the startup/navigation time for the page. **Recommendation:** As an alternative to using the convenient ICommand interface, consider putting event handlers in your code-behind and attaching them to the view events and call a command on your view-model when those events are raised. You'll also need to add extra code to disable the Button when the command is unavailable.
- It's popular in MVVM to create a Page with all possible configurations of the UI, then collapse parts of the tree by binding the Visibility property to properties in the VM. This adds unnecessarily to startup time and possibly to working set (because some parts of the tree may never become visible). **Recommendations:** Use the [x:Load attribute](#) or [x:DeferLoadStrategy attribute](#) feature to defer unnecessary portions of the tree out of startup. Also, create separate user controls for the different modes of the page and use code-behind to keep only the necessary controls loaded.

C++/CX recommendations

- **Use the latest version.** There are continual performance improvements made to the C++/CX compiler. Ensure your app is building using the latest toolset.
- **Disable RTTI (/GR-).** RTTI is on by default in the compiler so, unless your build environment switches it off, you're probably using it. RTTI has significant overhead, and unless your code has a deep dependency on it, you should turn it off. The XAML framework has no requirement that your code use RTTI.
- **Avoid heavy use of ppltasks.** Ppltasks are very convenient when calling async WinRT APIs, but they come with significant code size overhead. The C++/CX team is working on a language feature – await – that will provide much better performance. In the meantime, balance your use of ppltasks in the hot paths of your code.
- **Avoid use of C++/CX in the “business logic” of your app.** C++/CX is designed to be a convenient way to access WinRT APIs from C++ apps. It makes use of wrappers that have overhead. You should avoid C++/CX inside the business logic/model of your class, and reserve it for use at the boundaries between your code and WinRT.

Best practices for your app's startup performance

Article • 10/20/2022

Create Universal Windows Platform (UWP) apps with optimal startup times by improving the way you handle launch and activation.

Best practices for your app's startup performance

In part, users perceive whether your app is fast or slow based on how long it takes to start up. For the purposes of this topic, an app's startup time begins when the user starts the app, and ends when the user can interact with the app in some meaningful way. This section provides suggestions on how to get better performance out of your app when it starts.

Measuring your app's startup time

Be sure to start your app a few times before you actually measure its startup time. This gives you a baseline for your measurement and ensures that you're measuring as reasonably short a startup time as possible.

By the time your UWP app arrives on your customers' computers, your app has been compiled with the .NET Native toolchain. .NET Native is an ahead-of-time compilation technology that converts MSIL into natively-runnable machine code. .NET Native apps start faster, use less memory, and use less battery than their MSIL counterparts.

Applications built with .NET Native statically link in a custom runtime and the new converged .NET Core that can run on all devices, so they don't depend on the in-box .NET implementation. On your development machine, by default your app uses .NET Native if you're building it in "Release" mode, and it uses CoreCLR if you're building it in "Debug" mode. You can configure this in Visual Studio from the Build page in "Properties" (C#) or Compile->Advanced in "My Project" (VB). Look for a checkbox that says "Compile with .NET Native Toolchain".

Of course, you should take measurements that are representative of what the end user will experience. So, if you're not sure you're compiling your app to native code on your development machine, you could run the Native Image Generator (Ngen.exe) tool to precompile your app before you measure its startup time.

The following procedure describes how to run Ngen.exe to precompile your app.

To run Ngen.exe

1. Run your app at least one time to ensure that Ngen.exe detects it.
2. Open the **Task Scheduler** by doing one of the following:
 - Search for "Task Scheduler" from the start screen.
 - Run "taskschd.msc."
3. In the left-hand pane of **Task Scheduler**, expand **Task Scheduler Library**.
4. Expand **Microsoft**.
5. Expand **Windows**.
6. Select **.NET Framework**.
7. Select **.NET Framework NGEN 4.x** from the task list.

If you are using a 64-bit computer, there is also a **.NET Framework NGEN v4.x 64**.

If you are building a 64-bit app, select **.NET Framework NGEN v4.x 64**.

8. From the **Action** menu, click **Run**.

Ngen.exe precompiles all the apps on the machine that have been used and do not have native images. If there are a lot of apps that need to be precompiled, this can take a long time, but subsequent runs are much faster.

When you recompile your app, the native image is no longer used. Instead, the app is just-in-time compiled, which means that it is compiled as the app runs. You must rerun Ngen.exe to get a new native image.

Defer work as long as possible

To improve your app's startup time, do only the work that absolutely needs to be done to let the user start interacting with the app. This can be especially beneficial if you can delay loading additional assemblies. The common language runtime loads an assembly the first time it is used. If you can minimize the number of assemblies that are loaded, you might be able to improve your app's startup time and its memory consumption.

Do long-running work independently

Your app can be interactive even though there are parts of the app that aren't fully functional. For example, if your app displays data that takes a while to retrieve, you can make that code execute independently of the app's startup code by retrieving the data asynchronously. When the data is available, populate the app's user interface with the data.

Many of the Universal Windows Platform (UWP) APIs that retrieve data are asynchronous, so you will probably be retrieving data asynchronously anyway. For more info about asynchronous APIs, see [Call asynchronous APIs in C# or Visual Basic](#). If you do work that doesn't use asynchronous APIs, you can use the Task class to do long running work so that you don't block the user from interacting with the app. This will keep your app responsive to the user while the data loads.

If your app takes an especially long time to load part of its UI, consider adding a string in that area that says something like, "Getting latest data," so that your users know that the app is still processing.

Minimize startup time

All but the simplest apps require a perceivable amount of time to load resources, parse XAML, set up data structures, and run logic at activation. Here, we analyze the process of activation by breaking it into three phases. We also provide tips for reducing the time spent in each phase, and techniques for making each phase of your app's startup more palatable to the user.

The activation period is the time between the moment a user starts the app and the moment the app is functional. This is a critical time because it's a user's first impression of your app. They expect instant and continuous feedback from the system and apps. The system and the app are perceived to be broken or poorly designed when apps don't start quickly. Even worse, if an app takes too long to activate, the Process Lifetime Manager (PLM) might kill it, or the user might uninstall it.

Introduction to the stages of startup

Startup involves a number of moving pieces, and all of them need to be correctly coordinated for the best user experience. The following steps occur between your user clicking on your app tile and the application content being shown.

- The Windows shell starts the process and Main is called.
- The Application object is created.
 - (Project template) Constructor calls InitializeComponent, which causes App.xaml to be parsed and objects created.

- Application.OnLaunched event is raised.
 - (ProjectTemplate) App code creates a Frame and navigates to MainPage.
 - (ProjectTemplate) Mainpage constructor calls InitializeComponent which causes MainPage.xaml to be parsed and objects created.
 - ProjectTemplate) Window.Current.Activate() is called.
- XAML Platform runs the Layout pass including Measure & Arrange.
 - ApplyTemplate will cause control template content to be created for each control, which is typically the bulk of Layout time for startup.
- Render is called to create visuals for all the window contents.
- Frame is presented to the Desktop Windows Manager (DWM).

Do less in your Startup path

Keep your startup code path free from anything that is not needed for your first frame.

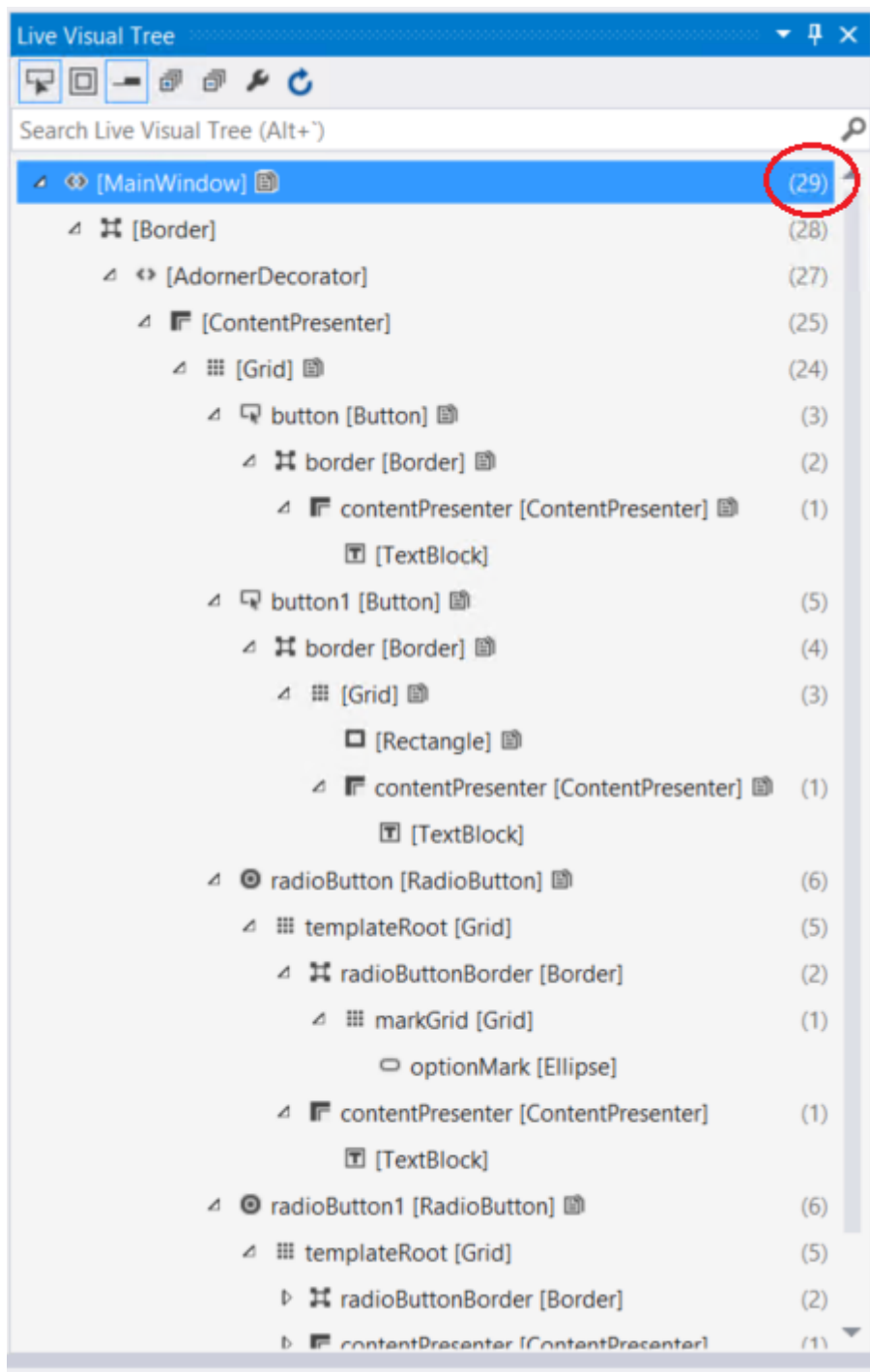
- If you have user dlls containing controls that are not needed during first frame, consider delay loading them.
- If you have a portion of your UI dependent on data from the cloud, then split that UI. First, bring up the UI that is not dependent on cloud data and asynchronously bring up the cloud-dependent UI. You should also consider caching data locally so that the application will work offline or not be affected by slow network connectivity.
- Show progress UI if your UI is waiting for data.
- Be cautious of app designs that involve a lot of parsing of configuration files, or UI that is dynamically generated by code.

Reduce element count

Startup performance in a XAML app is directly correlated to the number of elements you create during startup. The fewer elements you create, the less time your app will take to start up. As a rough benchmark, consider each element to take 1ms to create.

- Templates used in items controls can have the biggest impact, as they are repeated multiple times. See [ListView and GridView UI optimization](#).
- UserControls and control templates will be expanded, so those should also be taken into account.
- If you create any XAML that does not appear on the screen, then you should justify whether those pieces of XAML should be created during your startup.

The [Visual Studio Live Visual Tree](#) window shows the child element counts for each node in the tree.



Use deferral. Collapsing an element, or setting its opacity to 0, will not prevent the element from being created. Using `x:Load` or `x:DeferLoadStrategy`, you can delay the loading of a piece of UI, and load it when needed. This is good way to delay processing UI that is not visible during the startup screen, so that you can load it when needed, or as part of a set of delayed logic. To trigger the loading, you need only call `FindName` for the element. For an example and more information, see [x:Load attribute](#) and [x:DeferLoadStrategy attribute](#).

Virtualization. If you have list or repeater content in your UI then it's highly advised that you use UI virtualization. If list UI is not virtualized then you are paying the cost of creating all the elements up front, and that can slow down your startup. See [ListView and GridView UI optimization](#).

Application performance is not only about raw performance, it's also about perception. Changing the order of operations so that visual aspects occur first will commonly make the user feel like the application is faster. Users will consider the application loaded when the content is on the screen. Commonly, applications need to do multiple things as part of the startup, and not all of that is required to bring up the UI, so those should be delayed or prioritized lower than the UI.

This topic talks about the "first frame" which comes from animation/TV, and is a measure of how long until content is seen by the end user.

Improve startup perception

Let's use the example of a simple online game to identify each phase of startup and different techniques to give the user feedback throughout the process. For this example, the first phase of activation is the time between the user tapping the game's tile and the game starting to run its code. During this time, the system doesn't have any content to display to the user to even indicate that the correct game has started. But providing a splash screen gives that content to the system. The game then informs the user that the first phase of activation has completed by replacing the static splash screen with its own UI when it begins running code.

The second phase of activation encompasses creating and initializing structures critical for the game. If an app can quickly create its initial UI with the data available after the first phase of activation, then the second phase is trivial and you can display the UI immediately. Otherwise we recommend that the app display a loading page while it is initialized.

What the loading page looks like is up to you and it can be as simple as displaying a progress bar or a progress ring. The key point is that the app indicates that it is performing tasks before becoming responsive. In the case of the game, it would like to display its initial screen but that UI requires that some images and sounds be loaded from disk into memory. These tasks take a couple of seconds, so the app keeps the user informed by replacing the splash screen with a loading page, which shows a simple animation related to the theme of the game.

The third stage begins after the game has a minimal set of info to create an interactive UI, which replaces the loading page. At this point the only info available to the online game is the content that the app loaded from disk. The game can ship with enough content to create an interactive UI; but because it's an online game it won't be functional until it connects to the internet and downloads some additional info. Until it has all the info it needs to be functional, the user can interact with the UI, but features that need additional data from the web should give feedback that content is still

loading. It may take some time for an app to become fully functional, so it's important that functionality be made available as soon as possible.

Now that we have identified the three stages of activation in the online game, let's tie them to actual code.

Phase 1

Before an app starts, it needs to tell the system what it wants to display as the splash screen. It does so by providing an image and background color to the SplashScreen element in an app's manifest, as in the example. Windows displays this after the app begins activation.

XML

```
<Package ...>
  ...
  <Applications>
    <Application ...>
      <VisualElements ...>
        ...
        <SplashScreen Image="Images\splashscreen.png"
BackgroundColors="#000000" />
        ...
      </VisualElements>
    </Application>
  </Applications>
</Package>
```

For more info, see [Add a splash screen](#).

Use the app's constructor only to initialize data structures that are critical to the app. The constructor is called only the first time the app is run and not necessarily each time the app is activated. For example, the constructor isn't called for an app that has been run, placed in the background, and then activated via the search contract.

Phase 2

There are a number of reasons for an app to be activated, each of which you may want to handle differently. You can override [OnActivated](#), [OnCachedFileUpdaterActivated](#), [OnFileActivated](#), [OnFileOpenPickerActivated](#), [OnFileSavePickerActivated](#), [OnLaunched](#), [OnSearchActivated](#), and [OnShareTargetActivated](#) methods to handle each reason of activation. One of the things that an app must do in these methods is create a UI, assign it to [Window.Content](#), and then call [Window.Activate](#). At this point the splash screen is

replaced by the UI that the app created. This visual could either be loading screen or the app's actual UI if enough info is available at activation to create it.

C#

```
public partial class App : Application
{
    // A handler for regular activation.
    async protected override void OnLaunched(LaunchActivatedEventArgs args)
    {
        base.OnLaunched(args);

        // Asynchronously restore state based on generic launch.

        // Create the ExtendedSplash screen which serves as a loading page
        while the
        // reader downloads the section information.
        ExtendedSplash eSplash = new ExtendedSplash();

        // Set the content of the window to the extended splash screen.
        Window.Current.Content = eSplash;

        // Notify the Window that the process of activation is completed
        Window.Current.Activate();
    }

    // a different handler for activation via the search contract
    async protected override void OnSearchActivated(SearchActivatedEventArgs
args)
    {
        base.OnSearchActivated(args);

        // Do an asynchronous restore based on Search activation

        // the rest of the code is the same as the OnLaunched method
    }
}

partial class ExtendedSplash : Page
{
    // This is the UIElement that's the game's home page.
    private GameHomePage homePage;

    public ExtendedSplash()
    {
        InitializeComponent();
        homePage = new GameHomePage();
    }

    // Shown for demonstration purposes only.
    // This is typically autogenerated by Visual Studio.
    private void InitializeComponent()
    {

```

```
}  
}
```

VB

```
Partial Public Class App  
    Inherits Application  
  
    ' A handler for regular activation.  
    Protected Overrides Async Sub OnLaunched(ByVal args As  
LaunchActivatedEventArgs)  
        MyBase.OnLaunched(args)  
  
        ' Asynchronously restore state based on generic launch.  
  
        ' Create the ExtendedSplash screen which serves as a loading page  
while the  
        ' reader downloads the section information.  
        Dim eSplash As New ExtendedSplash()  
  
        ' Set the content of the window to the extended splash screen.  
        Window.Current.Content = eSplash  
  
        ' Notify the Window that the process of activation is completed  
        Window.Current.Activate()  
    End Sub  
  
    ' a different handler for activation via the search contract  
    Protected Overrides Async Sub OnSearchActivated(ByVal args As  
SearchActivatedEventArgs)  
        MyBase.OnSearchActivated(args)  
  
        ' Do an asynchronous restore based on Search activation  
  
        ' the rest of the code is the same as the OnLaunched method  
    End Sub  
End Class  
  
Partial Friend Class ExtendedSplash  
    Inherits Page  
  
    Public Sub New()  
        InitializeComponent()  
  
        ' Downloading the data necessary for  
        ' initial UI on a background thread.  
        Task.Run(Sub() DownloadData())  
    End Sub  
  
    Private Sub DownloadData()  
        ' Download data to populate the initial UI.  
  
        ' Create the first page.
```

```

    Dim firstPage As New MainPage()

    ' Add the data just downloaded to the first page

    ' Replace the loading page, which is currently
    ' set as the window's content, with the initial UI for the app
    Window.Current.Content = firstPage
End Sub

' Shown for demonstration purposes only.
' This is typically autogenerated by Visual Studio.
Private Sub InitializeComponent()
End Sub
End Class

```

Apps that display a loading page in the activation handler begin work to create the UI in the background. After that element has been created, its [FrameworkElement.Loaded](#) event occurs. In the event handler you replace the window's content, which is currently the loading screen, with the newly created home page.

It's critical that an app with an extended initialization period show a loading page. Aside from providing the user feedback about the activation process, the process will be terminated if [Window.Activate](#) is not called within 15 seconds of the start of the activation process.

C#

```

partial class GameHomePage : Page
{
    public GameHomePage()
    {
        InitializeComponent();

        // add a handler to be called when the home page has been loaded
        this.Loaded += ReaderHomePageLoaded;

        // load the minimal amount of image and sound data from disk
        // necessary to create the home page.
    }

    void ReaderHomePageLoaded(object sender, RoutedEventArgs e)
    {
        // set the content of the window to the home page now that it's
        // ready to be displayed.
        Window.Current.Content = this;
    }

    // Shown for demonstration purposes only.
    // This is typically autogenerated by Visual Studio.
    private void InitializeComponent()
    {

```

```
}  
}
```

VB

```
Partial Friend Class GameHomePage  
Inherits Page  
  
Public Sub New()  
    InitializeComponent()  
  
    ' add a handler to be called when the home page has been loaded  
    AddHandler Me.Loaded, AddressOf ReaderHomePageLoaded  
  
    ' load the minimal amount of image and sound data from disk  
    necessary to create the home page.  
End Sub  
  
Private Sub ReaderHomePageLoaded(ByVal sender As Object, ByVal e As  
RoutedEventArgs)  
    ' set the content of the window to the home page now that it's ready  
    to be displayed.  
    Window.Current.Content = Me  
End Sub  
  
' Shown for demonstration purposes only.  
' This is typically autogenerated by Visual Studio.  
Private Sub InitializeComponent()  
End Sub  
End Class
```

For an example of using extended splash screens, see [Splash screen sample](#).

Phase 3

Just because the app displayed the UI doesn't mean it is completely ready for use. In the case of our game, the UI is displayed with placeholders for features that require data from the internet. At this point the game downloads the additional data needed to make the app fully functional and progressively enables features as data is acquired.

Sometimes much of the content needed for activation can be packaged with the app. Such is the case with a simple game. This makes the activation process quite simple. But many programs (such as news readers and photo viewers) must pull information from the web to become functional. This data can be large and take a fair amount of time to download. How the app gets this data during the activation process can have a huge impact on the perceived performance of an app.

You could display a loading page, or worse, a splash screen, for minutes if an app tried to download an entire data set it needs for functionality in phase one or two of activation. This makes an app look like it's hung or cause it to be terminated by the system. We recommend that an app download the minimal amount of data to show an interactive UI with placeholder elements in phase 2 and then progressively load data, which replaces the placeholder elements, in phase 3. For more info on dealing with data, see [Optimize ListView and GridView](#).

How exactly an app reacts to each phase of startup is completely up to you, but providing the user as much feedback as possible (splash screen, loading screen, UI while data loads) makes the user feel as though an app, and the system as a whole, are fast.

Minimize managed assemblies in the startup path

Reusable code often comes in the form of modules (DLLs) included in a project. Loading these modules requires accessing the disk, and as you can imagine, the cost of doing so can add up. This has the greatest impact on cold startup, but it can have an impact on warm startup, too. In the case of C# and Visual Basic, the CLR tries to delay that cost as much as possible by loading assemblies on demand. That is, the CLR doesn't load a module until an executed method references it. So, reference only assemblies that are necessary to the launch of your app in startup code so that the CLR doesn't load unnecessary modules. If you have unused code paths in your startup path that have unnecessary references, you can move these code paths to other methods to avoid the unnecessary loads.

Another way to reduce module loads is to combine your app modules. Loading one large assembly typically takes less time than loading two small ones. This is not always possible, and you should combine modules only if it doesn't make a material difference to developer productivity or code reusability. You can use tools such as [PerfView](#) or the [Windows Performance Analyzer \(WPA\)](#) to find out what modules are loaded on startup.

Make smart web requests

You can dramatically improve the loading time of an app by packaging its contents locally, including XAML, images, and any other files important to the app. Disk operations are faster than network operations. If an app needs a particular file at initialization, you can reduce the overall startup time by loading it from disk instead of retrieving it from a remote server.

Journal and Cache Pages Efficiently

The Frame control provides navigation features. It offers navigation to a Page (Navigate method), navigation journaling (BackStack/ForwardStack properties, GoForward/GoBack method), Page caching (Page.NavigationCacheMode), and serialization support (GetNavigationState method).

The performance to be aware of with Frame is primarily around the journaling and page caching.

Frame journaling. When you navigate to a page with `Frame.Navigate()`, a `PageStackEntry` for the current page is added to `Frame.BackStack` collection. `PageStackEntry` is relatively small, but there's no built-in limit to the size of the `BackStack` collection. Potentially, a user could navigate in a loop and grow this collection indefinitely.

The `PageStackEntry` also includes the parameter that was passed to the `Frame.Navigate()` method. It's recommended that that parameter be a primitive serializable type (such as an `int` or `string`), in order to allow the `Frame.GetNavigationState()` method to work. But that parameter could potentially reference an object that accounts for more significant amounts of working set or other resources, making each entry in the `BackStack` that much more expensive. For example, you could potentially use a `StorageFile` as a parameter, and consequently the `BackStack` is keeping an indefinite number of files open.

Therefore it's recommended to keep the navigation parameters small, and to limit the size of the `BackStack`. The `BackStack` is a standard vector (`IList` in C#, `Platform::Vector` in C++/CX), and so can be trimmed simply by removing entries.

Page caching. By default, when you navigate to a page with the `Frame.Navigate` method, a new instance of the page is instantiated. Similarly, if you then navigate back to the previous page with `Frame.GoBack`, a new instance of the previous page is allocated.

Frame, though, offers an optional page cache that can avoid these instantiations. To get a page put into the cache, use the `Page.NavigationCacheMode` property. Setting that mode to `Required` will force the page to be cached, setting it to `Enabled` will allow it to be cached. By default the cache size is 10 pages, but this can be overridden with the `Frame.CacheSize` property. All `Required` pages will be cached, and if there are fewer than `CacheSize Required` pages, `Enabled` pages can be cached as well.

Page caching can help performance by avoiding instantiations, and therefore improving navigation performance. Page caching can hurt performance by over-caching and

therefore impacting working set.

Therefore it's recommended to use page caching as appropriate for your application. For example, say you have an app that shows a list of items in a Frame, and when you tap on an item, it navigates the frame to a detail page for that item. The list page should probably be set to cache. If the detail page is the same for all items, it should probably be cached as well. But if the detail page is more heterogeneous, it might be better to leave caching off.

Optimize animations, media, and images

Article • 10/20/2022

Create Universal Windows Platform (UWP) apps with smooth animations, high frame rate, and high-performance media capture and playback.

Make animations smooth

A key aspect of UWP apps is smooth interactions. This includes touch manipulations that "stick to your finger," smooth transitions and animations, and small motions that provide input feedback. In the XAML framework there is a thread called the composition thread that is dedicated to the composition and animation of an app's visual elements. Because the composition thread is separate from UI thread (the thread that runs framework and developer code), apps can achieve a consistent frame rate and smooth animations regardless of complicated layout passes or extended calculations. This section shows how to use the composition thread to keep an app's animations buttery smooth. For more info about animations, see [Animations overview](#). To learn about increasing an app's responsiveness while performing intensive computations, see [Keep the UI thread responsive](#).

Use independent instead of dependent animations

Independent animations can be calculated from beginning to end at the time of creation because changes to the property being animated don't affect rest of the objects in a scene. Independent animations can therefore run on the composition thread instead of the UI thread. This guarantees that they remain smooth because the composition thread is updated at a consistent cadence.

All of these types of animations are guaranteed to be independent:

- Object animations using key frames
- Zero-duration animations
- Animations to the [Canvas.Left](#) and [Canvas.Top](#) properties
- Animations to the [UIElement.Opacity](#) property
- Animations to properties of type [Brush](#) when targeting the [SolidColorBrush.Color](#) subproperty

- Animations to the following **UIElement** properties when targeting subproperties of these return value types:
 - **RenderTransform**
 - **Transform3D**
 - **Projection**
 - **Clip**

Dependent animations affect layout, which therefore cannot be calculated without extra input from the UI thread. Dependent animations include modifications to properties like **Width** and **Height**. By default, dependent animations are not run and require an opt-in from the app developer. When enabled, they run smoothly if the UI thread remains unblocked, but they begin to stutter if the framework or app is doing a lot of other work on the UI thread.

Almost all animations in the XAML framework are independent by default, but there are some actions that you can take to disable this optimization. Beware of these scenarios particularly:

- Setting the **EnableDependentAnimation** property to allow a dependent animation to run on the UI thread. Convert these animations into an independent version. For example animate **ScaleTransform.ScaleX** and **ScaleTransform.ScaleY** instead of the **Width** and **Height** of an object. Don't be afraid to scale objects like images and text. The framework applies bilinear scaling only while the **ScaleTransform** is being animated. The image/text will be rerasterized at the final size to ensure it's always clear.
- Making per frame updates, which are effectively dependent animations. An example of this is applying transformations in the handler of the **CompositonTarget.Rendering** event.
- Running any animation considered independent in an element with the **CacheMode** property set to **BitmapCache**. This is considered dependent because the cache must be re-rasterized for each frame.

Don't animate a **WebView** or **MediaPlayerElement**

Web content within a **WebView** control is not directly rendered by the XAML framework and it requires extra work to be composed with the rest of the scene. This extra work adds up when animating the control around the screen and can potentially introduce synchronization issues (for example, the HTML content might not move in sync with the rest of the XAML content on the page). When you need to animate a **WebView** control, swap it with a **WebViewBrush** for the duration of the animation.

Animating a [MediaPlayerElement](#) is a similarly bad idea. Beyond the performance detriment, it can cause tearing or other artifacts in the video content being played.

Note The recommendations in this article for [MediaPlayerElement](#) also apply to [MediaElement](#). [MediaPlayerElement](#) is only available in Windows 10, version 1607, so if you are creating an app for a previous version of Windows you need to use [MediaElement](#).

Use infinite animations sparingly

Most animations execute for a specified amount of time, but setting the [Timeline.Duration](#) property to Forever allows an animation to run indefinitely. We recommend minimizing the use of infinite animations because they continually consume CPU resources and can prevent the CPU from going into a low power or idle state, causing it to run out of power more quickly.

Adding a handler for [CompositionTarget.Rendering](#) is similar to running an infinite animation. Normally the UI thread is active only when there is work to do, but adding handler for this event forces it to run every frame. Remove the handler when there is no work to be done and reregister it when it's needed again.

Use the animation library

The [Windows.UI.Xaml.Media.Animation](#) namespace includes a library of high-performance, smooth animations that have a look and feel consistent with other Windows animations. The relevant classes have "Theme" in their name, and are described in [Animations overview](#). This library supports many common animation scenarios, such as animating the first view of the app and creating state and content transitions. We recommend using this animation library whenever possible to increase performance and consistency for UWP UI.

Note The animation library can't animate all possible properties. For XAML scenarios where the animation library doesn't apply, see [Storyboarded animations](#).

Animate CompositeTransform3D properties independently

You can animate each property of a [CompositeTransform3D](#) independently, so apply only the animations you need. For examples and more info, see [UIElement.Transform3D](#). For more info about animating transforms, see [Storyboarded animations](#) and [Key-frame and easing function animations](#).

Optimize media resources

Audio, video, and images are compelling forms of content that the majority of apps use. As media capture rates increase and content moves from standard definition to high definition the amount of resources need to store, decode, and play back this content increases. The XAML framework builds on the latest features added to the UWP media engines so apps get these improvements for free. Here we explain some additional tricks that allow you to get the most out media in your UWP app.

Release media streams

Media files are some of the most common and expensive resources apps typically use. Because media file resources can greatly increase the size of your app's memory footprint, you must remember to release the handle to media as soon as the app is finished using it.

For example, if your app working with a [RandomAccessStream](#) or an [InputStream](#) object, be sure to call the close method on the object when your app has finished using it, to release the underlying object.

Display full screen video playback when possible

In UWP apps, always use the [IsFullWindow](#) property on the [MediaPlayerElement](#) to enable and disable full window rendering. This insures system level optimizations are used during media playback.

The XAML framework can optimize the display of video content when it is the only thing being rendered, resulting in an experience that uses less power and yields higher frame rates. For most efficient media playback set the size of a **MediaPlayerElement** to be the width and height of the screen and don't display other XAML elements

There are legitimate reasons to overlay XAML elements on a **MediaPlayerElement** that takes up the full width and height of the screen, for example closed captions or momentary transport controls. Make sure to hide these elements (set `Visibility="Collapsed"`) when they are not needed to put media playback back into its most efficient state.

Display deactivation and conserving power

To prevent the display from be deactivating when user action is no longer detected, such as when an app is playing video, you can call [DisplayRequest.RequestActive](#).

To conserve power and battery life, you should call [DisplayRequest.RequestRelease](#) to release the display request as soon as it is no longer required.

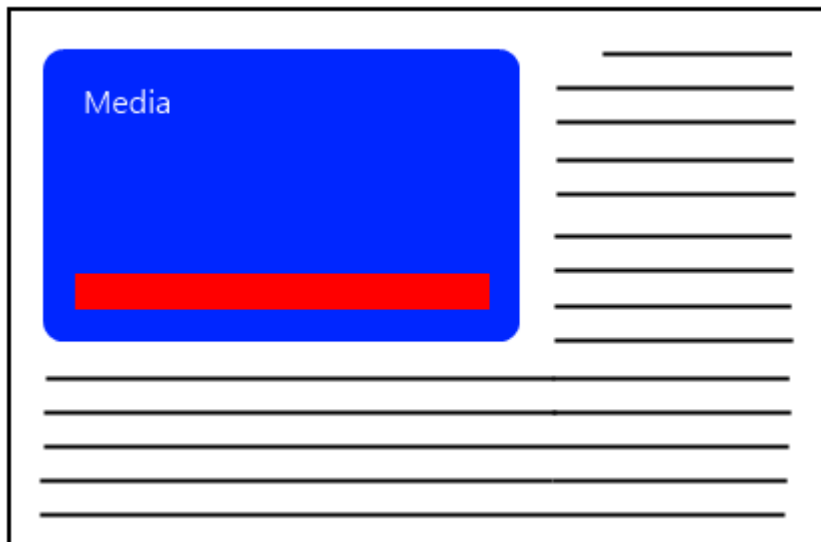
Here are some situations when you should release the display request:

- Video playback is paused, for example by user action, buffering, or adjustment due to limited bandwidth.
- Playback stops. For example, the video is done playing or the presentation is over.
- A playback error has occurred. For example, network connectivity issues or a corrupted file.

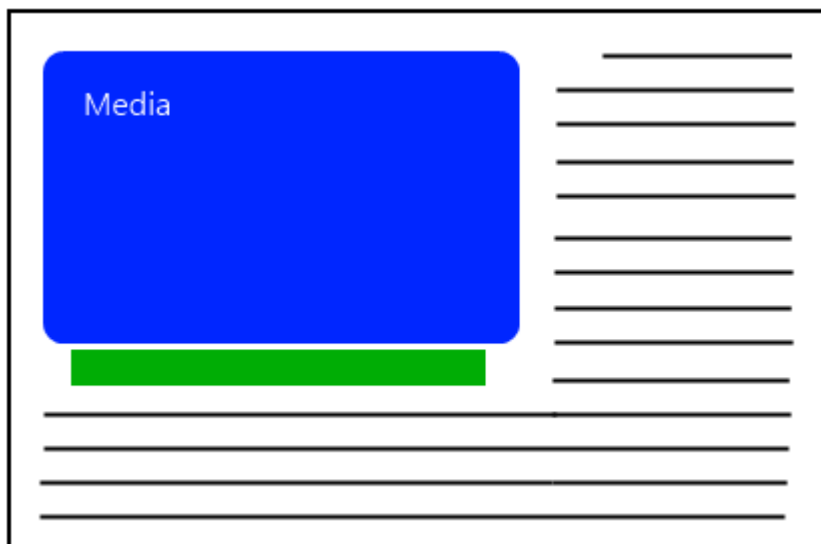
Put other elements to the side of embedded video

Often apps offer an embedded view where video is played within a page. Now you obviously lost the full screen optimization because the [MediaPlayerElement](#) is not the size of the page and there are other XAML objects drawn. Beware of unintentionally entering this mode by drawing a border around a **MediaPlayerElement**.

Don't draw XAML elements on top of video when it's in embedded mode. If you do, the framework is forced to do a little extra work to compose the scene. Placing transport controls below an embedded media element instead of on top of the video is a good example of optimizing for this situation. In this image, the red bar indicates a set of transport controls (play, pause, stop, etc.).



Don't place these controls on top of media that is not full screen. Instead place the transport controls somewhere outside of the area where the media is being rendered. In the next image, the controls are placed below the media.



Delay setting the source for a MediaPlayerElement

Media engines are expensive objects and the XAML framework delays loading dlls and creating large objects as long as possible. The [MediaPlayerElement](#) is forced to do this work after its source is set via the [Source](#) property. Setting this when the user is really ready to play media delays the majority of the cost associated with the [MediaPlayerElement](#) as long as possible.

Set MediaPlayerElement.PosterSource

Setting [MediaPlayerElement.PosterSource](#) enables XAML to release some GPU resources that would have otherwise been used. This API allows an app to use as little memory as possible.

Improve media scrubbing

Scrubbing is always a tough task for media platforms to make really responsive. Generally people accomplish this by changing the value of a Slider. Here are a couple tips on how to make this as efficient as possible:

- Update the value of [Slider](#) based on a timer that queries the [Position](#) on the [MediaPlayerElement.MediaPlayer](#). Make sure to use a reasonable update frequency for your timer. The [Position](#) property only updates every 250 millisecond during playback.
- The size of the step frequency on the Slider must scale with the length of the video.
- Subscribe to the [PointerPressed](#), [PointerMoved](#), [PointerReleased](#) events on the slider to set the [PlaybackRate](#) property to 0 when the user drags the thumb of the slider.

- In the [PointerReleased](#) event handler, manually set the media position to the slider position value to achieve optimal thumb snapping while scrubbing.

Match video resolution to device resolution

Decoding video takes a lot of memory and GPU cycles, so choose a video format close to the resolution it will be displayed at. There is no point in using the resources to decode 1080 video if it's going to get scaled down to a much smaller size. Many apps don't have the same video encoded at different resolutions; but if it is available, use an encoding that is close to the resolution at which it will be displayed.

Choose recommended formats

Media format selection can be a sensitive topic and is often driven by business decisions. From a UWP performance perspective, we recommend H.264 video as the primary video format and AAC and MP3 as the preferred audio formats. For local file playback, MP4 is the preferred file container for video content. H.264 decoding is accelerated through most recent graphics hardware. Also, although hardware acceleration for VC-1 decoding is broadly available, for a large set of graphics hardware on the market, the acceleration is limited in many cases to a partial acceleration level (or IDCT level), rather than a full-steam level hardware offload (i.e. VLD mode).

If you have full control of the video content generation process, you must figure out how to keep a good balance between compression efficiency and GOP structure. Relatively smaller GOP size with B pictures can increase the performance in seeking or trick modes.

When including short, low-latency audio effects, for example in games, use WAV files with uncompressed PCM data to reduce processing overhead that is typical for compressed audio formats.

Optimize image resources

Scale images to the appropriate size

Images are captured at very high resolutions, which can lead to apps using more CPU when decoding the image data and more memory after it's loaded from disk. But there's no sense decoding and saving a high-resolution image in memory only to display it smaller than its native size. Instead, create a version of the image at the exact size it will be drawn on-screen using the [DecodePixelWidth](#) and [DecodePixelHeight](#) properties.

Don't do this:

XAML

```
<Image Source="ms-appx:///Assets/highresCar.jpg"
        Width="300" Height="200"/>    <!-- BAD CODE DO NOT USE.-->
```

Instead, do this:

XAML

```
<Image>
    <Image.Source>
        <BitmapImage UriSource="ms-appx:///Assets/highresCar.jpg"
                      DecodePixelWidth="300" DecodePixelHeight="200"/>
    </Image.Source>
</Image>
```

The units for **DecodePixelWidth** and **DecodePixelHeight** are by default physical pixels. The **DecodePixelType** property can be used to change this behavior: setting **DecodePixelType** to **Logical** results in the decode size automatically accounting for the system's current scale factor, similar to other XAML content. It would therefore be generally appropriate to set **DecodePixelType** to **Logical** if, for instance, you want **DecodePixelWidth** and **DecodePixelHeight** to match the Height and Width properties of the Image control the image will be displayed in. With the default behavior of using physical pixels, you must account for the system's current scale factor yourself; and you should listen for scale change notifications in case the user changes their display preferences.

If DecodePixelWidth/Height are explicitly set larger than the image will be displayed on-screen then the app will unnecessarily use extra memory—up to 4 bytes per pixel—which quickly becomes expensive for large images. The image will also be scaled down using bilinear scaling which could cause it to appear blurry for large scale factors.

If DecodePixelWidth/DecodePixelHeight are explicitly set smaller than the image will be displayed on screen then it will be scaled up and could appear pixelated.

In some cases where an appropriate decode size cannot be determined ahead of time, you should defer to XAML's automatic right-size-decoding which will make a best effort attempt to decode the image at the appropriate size if an explicit DecodePixelWidth/DecodePixelHeight is not specified.

You should set an explicit decode size if you know the size of the image content ahead of time. You should also in conjunction set **DecodePixelType** to **Logical** if the supplied

decode size is relative to other XAML element sizes. For example, if you explicitly set the content size with `Image.Width` and `Image.Height`, you could set `DecodePixelType` to `DecodePixelType.Logical` to use the same logical pixel dimensions as an `Image` control and then explicitly use `BitmapImage.DecodePixelWidth` and/or `BitmapImage.DecodePixelHeight` to control the size of the image to achieve potentially large memory savings.

Note that `Image.Stretch` should be considered when determining the size of the decoded content.

Right-sized decoding

In the event that you don't set an explicit decode size, XAML will make a best effort attempt to save memory by decoding an image to the exact size it will appear on-screen according to the containing page's initial layout. You're advised to write your application in such a way as to make use of this feature when possible. This feature will be disabled if any of the following conditions are met.

- The `BitmapImage` is connected to the live XAML tree after setting the content with `SetSourceAsync` or `UriSource`.
- The image is decoded using synchronous decoding such as `SetSource`.
- The image is hidden via setting `Opacity` to 0 or `Visibility` to `Collapsed` on the host image element or brush or any parent element.
- The image control or brush uses a `Stretch` of `None`.
- The image is used as a `NineGrid`.
- `CacheMode="BitmapCache"` is set on the image element or on any parent element.
- The image brush is non-rectangular (such as when applied to a shape or to text).

In the above scenarios, setting an explicit decode size is the only way to achieve memory savings.

You should always attach a `BitmapImage` to the live tree before setting the source. Any time an image element or brush is specified in markup, this will automatically be the case. Examples are provided below under the heading "Live tree examples". You should always avoid using `SetSource` and instead use `SetSourceAsync` when setting a stream source. And it's a good idea to avoid hiding image content (either with zero opacity or with collapsed visibility) while waiting for the `ImageOpened` event to be raised. Doing this is a judgment call: you won't benefit from automatic right-sized decoding if it's done. If your app must hide image content initially then it should also set the decode size explicitly if possible.

Live tree examples

Example 1 (good)—Uniform Resource Identifier (URI) specified in markup.

XAML

```
<Image x:Name="myImage" UriSource="Assets/cool-image.png"/>
```

Example 2 markup—URI specified in code-behind.

XAML

```
<Image x:Name="myImage"/>
```

Example 2 code-behind (good)—connecting the BitmapImage to the tree before setting its UriSource.

C#

```
var bitmapImage = new BitmapImage();  
myImage.Source = bitmapImage;  
bitmapImage.UriSource = new Uri("ms-appx:///Assets/cool-image.png",  
UriKind.RelativeOrAbsolute);
```

Example 2 code-behind (bad)—setting the BitmapImage's UriSource before connecting it to the tree.

C#

```
var bitmapImage = new BitmapImage();  
bitmapImage.UriSource = new Uri("ms-appx:///Assets/cool-image.png",  
UriKind.RelativeOrAbsolute);  
myImage.Source = bitmapImage;
```

Caching optimizations

Caching optimizations are in effect for images that use [UriSource](#) to load content from an app package or from the web. The URI is used to uniquely identify the underlying content, and internally the XAML framework will not download or decode the content multiple times. Instead, it will use the cached software or hardware resources to display the content multiple times.

The exception to this optimization is if the image is displayed multiple times at different resolutions (which can be specified explicitly or through automatic right-sized decoding). Each cache entry also stores the resolution of the image, and if XAML cannot

find an image with a source URI that matches the required resolution then it will decode a new version at that size. It will not, however, download the encoded image data again.

Consequently, you should embrace using [UriSource](#) when loading images from an app package, and avoid using a file stream and [SetSourceAsync](#) when it's not required.

Images in virtualized panels (ListView, for instance)

If an image is removed from the tree—because the app explicitly removed it, or because it's in a modern virtualized panel and was implicitly removed when scrolled out of view—then XAML will optimize memory usage by releasing the hardware resources for the image since they are no longer required. The memory is not released immediately, but rather is released during the frame update that occurs after one second of the image element no longer being in the tree.

Consequently, you should strive to use modern virtualized panels to host lists of image content.

Software-rasterized images

When an image is used for a non-rectangular brush or for a [NineGrid](#), the image will use a software rasterization path, which will not scale images at all. Additionally, it must store a copy of the image in both software and hardware memory. For instance, if an image is used as a brush for an ellipse then the potentially large full image will be stored twice internally. When using [NineGrid](#) or a non-rectangular brush, then, your app should pre-scale its images to approximately the size they will be rendered at.

Background thread image-loading

XAML has an internal optimization that allows it to decode the contents of an image asynchronously to a surface in hardware memory without requiring an intermediate surface in software memory. This reduces peak memory usage and rendering latency. This feature will be disabled if any of the following conditions are met.

- The image is used as a [NineGrid](#).
- `CacheMode="BitmapCache"` is set on the image element or on any parent element.
- The image brush is non-rectangular (such as when applied to a shape or to text).

SoftwareBitmapSource

The [SoftwareBitmapSource](#) class exchanges interoperable uncompressed images between different WinRT namespaces such as [BitmapDecoder](#), camera APIs, and XAML. This class obviates an extra copy that would typically be necessary with [WriteableBitmap](#), and that helps reduce peak memory and source-to-screen latency.

The [SoftwareBitmap](#) that supplies source information can also be configured to use a custom [IWICBitmap](#) to provide a reloadable backing store that allows the app to re-map memory as it sees fit. This is an advanced C++ use case.

Your app should use [SoftwareBitmap](#) and [SoftwareBitmapSource](#) to interoperate with other WinRT APIs that produce and consume images. And your app should use [SoftwareBitmapSource](#) when loading uncompressed image data instead of using [WriteableBitmap](#).

Use GetThumbnailAsync for thumbnails

One use case for scaling images is creating thumbnails. Although you could use [DecodePixelWidth](#) and [DecodePixelHeight](#) to provide small versions of images, UWP provides even more efficient APIs for retrieving thumbnails. [GetThumbnailAsync](#) provides the thumbnails for images that have the file system already cached. This provides even better performance than the XAML APIs because the image doesn't need to be opened or decoded.

C#

```
FileOpenPicker picker = new FileOpenPicker();
picker.FileTypeFilter.Add(".bmp");
picker.FileTypeFilter.Add(".jpg");
picker.FileTypeFilter.Add(".jpeg");
picker.FileTypeFilter.Add(".png");
picker.SuggestedStartLocation = PickerLocationId.PicturesLibrary;

StorageFile file = await picker.PickSingleFileAsync();

StorageItemThumbnail fileThumbnail = await
file.GetThumbnailAsync(ThumbnailMode.SingleItem, 64);

BitmapImage bmp = new BitmapImage();
bmp.SetSource(fileThumbnail);

Image img = new Image();
img.Source = bmp;
```

VB

```
Dim picker As New FileOpenPicker()
picker.FileTypeFilter.Add(".bmp")
picker.FileTypeFilter.Add(".jpg")
picker.FileTypeFilter.Add(".jpeg")
picker.FileTypeFilter.Add(".png")
picker.SuggestedStartLocation = PickerLocationId.PicturesLibrary

Dim file As StorageFile = Await picker.PickSingleFileAsync()

Dim fileThumbnail As StorageItemThumbnail = Await
file.GetThumbnailAsync(ThumbnailMode.SingleItem, 64)

Dim bmp As New BitmapImage()
bmp.SetSource(fileThumbnail)

Dim img As New Image()
img.Source = bmp
```

Decode images once

To prevent images from being decoded more than once, assign the [Image.Source](#) property from an Uri rather than using memory streams. The XAML framework can associate the same Uri in multiple places with one decoded image, but it cannot do the same for multiple memory streams that contain the same data and creates a different decoded image for each memory stream.

Optimize suspend/resume

Article • 10/20/2022

Create Universal Windows Platform (UWP) apps that streamline their use of the process lifetime system to resume efficiently after suspension or termination.

Launch

When reactivating an app following suspend/terminate, check to see if a long time has elapsed. If so, consider returning to the main landing page of the app instead of showing the user stale data. This will also result in faster startup.

During activation, always check the `PreviousExecutionState` of the event args parameter (for example, for launched activations check `LaunchActivatedEventArgs.PreviousExecutionState`). If the value is `ClosedByUser` or `NotRunning`, don't waste time restoring previously saved state. In this case, the right thing is to provide a fresh experience – and it will result in faster startup.

Instead of eagerly restoring previously saved state, consider keep track of that state, and only restoring it on demand. For example, consider a situation where your app was previously suspended, saved state for 3 pages, and was then terminated. Upon relaunch, if you decide to return the user to the 3rd page, do not eagerly restore the state for the first 2 pages. Instead, hold on to this state and only use it once you know you need it.

While running

As a best practice, don't wait for the suspend event and then persist a large amount of state. Instead, your application should incrementally persist smaller amounts of state as it runs. This is especially important for large apps that are at risk of running out of time during suspend if they try to save everything at once.

However, you need to find a good balance between incremental saving and performance of your app while running. A good tradeoff is to incrementally keep track of the data that has changed (and therefore needs to be saved) – and use the suspend event to actually save that data (which is faster than saving all data or examining the entire state of app to decide what to save).

Don't use the window `Activated` or `VisibilityChanged` events to decide when to save state. When the user switches away from your app, the window is deactivated, but the system waits a short amount of time (about 10 seconds) before suspending the app.

This is to give a more responsive experience in case the user switches back to your app rapidly. Wait for the suspend event before running suspend logic.

Suspend

During suspend, reduce the footprint of your app. If your app uses less memory while suspended, the overall system will be more responsive and fewer suspended apps (including yours) will be terminated. However, balance this with the need for snappy resumes: don't reduce footprint so much that resume slows down considerably while your app reloads lots of data into memory.

For managed apps, the system will run a garbage collection pass after the app's suspend handlers complete. Make sure to take advantage of this by releasing references to objects that will help reduce the app's footprint while suspended.

Ideally, your app will finish with suspend logic in less than 1 second. The faster you can suspend, the better – that will result in a snappier user experience for other apps and parts of the system. If you must, your suspend logic can take up to 5 seconds on desktop devices or 10 seconds on mobile devices. If those times are exceeded, your app will be abruptly terminated. You don't want this to happen – because if it does, when the user switches back to your app, a new process will be launched and the experience will feel much slower compared to resuming a suspended app.

Resume

Most apps don't need to do anything special when resumed, so typically you won't handle this event. Some apps use resume to restore connections that were closed during suspend, or to refresh data that may be stale. Instead of doing this kind of work eagerly, design your app to initiate these activities on demand. This will result in a faster experience when the user switches back to a suspended app, and ensures that you're only doing work the user really needs.

Avoid unnecessary termination

The UWP process lifetime system can suspend or terminate an app for a variety of reasons. This process is designed to quickly return an app to the state it was in before it was suspended or terminated. When done well, the user won't be aware that the app ever stopped running. Here are a few tricks that your UWP app can use to help the system streamline transitions in an app's lifetime.

An app can be suspended when the user moves it to the background or when the system enters a low power state. When the app is being suspended, it raises the suspending event and has up to 5 seconds to save its data. If the app's suspending event handler doesn't complete within 5 seconds, the system assumes the app has stopped responding and terminates it. A terminated app has to go through the long startup process again instead of being immediately loaded into memory when a user switches to it.

Serialize only when necessary

Many apps serialize all their data on suspension. If you only need to store a small amount of app settings data, however, you should use the [LocalSettings](#) store instead of serializing the data. Use serialization for larger amounts of data and for non-settings data.

When you do serialize your data, you should avoid reserializing if it hasn't changed. It takes extra time to serialize and save the data, plus extra time to read and deserialize it when the app is activated again. Instead, we recommend that the app determine if its state has actually changed, and if so, serialize and deserialize only the data that changed. A good way to ensure that this happens is to periodically serialize data in the background after it changes. When you use this technique, everything that needs to be serialized at suspension has already been saved so there is no work to do and an app suspends quickly.

Serializing data in C# and Visual Basic

The available choices of serialization technology for .NET apps are the [System.Xml.Serialization.XmlSerializer](#), [System.Runtime.Serialization.DataContractSerializer](#), and [System.Runtime.Serialization.Json.DataContractJsonSerializer](#) classes.

From a performance perspective, we recommend using the [XmlSerializer](#) class. The [XmlSerializer](#) has the lowest serialization and deserialization times, and maintains a low memory footprint. The [XmlSerializer](#) has few dependencies on the .NET framework; this means that compared with the other serialization technologies, fewer modules need to be loaded into your app to use the [XmlSerializer](#).

[DataContractSerializer](#) makes it easier to serialize custom classes, although it has a larger performance impact than [XmlSerializer](#). If you need better performance, consider switching. In general, you should not load more than one serializer, and you should prefer [XmlSerializer](#) unless you need the features of another serializer.

Reduce memory footprint

The system tries to keep as many suspended apps in memory as possible so that users can quickly and reliably switch between them. When an app is suspended and stays in the system's memory, it can quickly be brought to the foreground for the user to interact with, without having to display a splash screen or perform a lengthy load operation. If there aren't enough resources to keep an app in memory, the app is terminated. This makes memory management important for two reasons:

- Freeing as much memory as possible at suspension minimizes the chances that your app is terminated because of lack of resources while it's suspended.
- Reducing the overall amount of memory your app uses reduces the chances that other apps are terminated while they are suspended.

Release resources

Certain objects, such as files and devices, occupy a large amount of memory. We recommend that during suspension, an app release handles to these objects and recreate them when needed. This is also a good time to purge any caches that won't be valid when the app is resumed. An additional step the XAML framework runs on your behalf for C# and Visual Basic apps is garbage collection if it is necessary. This ensures any objects no longer referenced in app code are released.

Resume quickly

A suspended app can be resumed when the user moves it to the foreground or when the system comes out of a low power state. When an app is resumed from the suspended state, it continues from where it was when it was suspended. No app data is lost because it was stored in memory, even if the app was suspended for a long period of time.

Most apps don't need to handle the [Resuming](#) event. When your app is resumed, variables and objects have the exact same state they had when the app was suspended. Handle the **Resuming** event only if you need to update data or objects that might have changed between the time your app was suspended and when it was resumed such as: content (for example, update feed data), network connections that may have gone stale, or if you need to reacquire access to a device (for example, a webcam).

Related topics

- [Guidelines for app suspend and resume](#)

Optimize file access

Article • 10/20/2022

Create Universal Windows Platform (UWP) apps that access the file system efficiently, avoiding performance issues due to disk latency and memory/CPU cycles.

When you want to access a large collection of files and you want to access property values other than the typical Name, FileType, and Path properties, access them by creating [QueryOptions](#) and calling [SetPropertyPrefetch](#). The [SetPropertyPrefetch](#) method can dramatically improve the performance of apps that display a collection of items obtained from the file system, such as a collection of images. The next set of examples shows a few ways to access multiple files.

The first example uses [Windows.Storage.StorageFolder.GetFilesAsync](#) to retrieve the name info for a set of files. This approach provides good performance, because the example accesses only the name property.

C#

```
StorageFolder library = Windows.Storage.KnownFolders.PicturesLibrary;
IReadOnlyList<StorageFile> files = await
library.GetFilesAsync(Windows.Storage.Search.CommonFileQuery.OrderByDate);

for (int i = 0; i < files.Count; i++)
{
    // do something with the name of each file
    string fileName = files[i].Name;
}
```

VB

```
Dim library As StorageFolder = Windows.Storage.KnownFolders.PicturesLibrary
Dim files As IReadOnlyList(Of StorageFile) =
    Await
library.GetFilesAsync(Windows.Storage.Search.CommonFileQuery.OrderByDate)

For i As Integer = 0 To files.Count - 1
    ' do something with the name of each file
    Dim fileName As String = files(i).Name
Next i
```

The second example uses [Windows.Storage.StorageFolder.GetFilesAsync](#) and then retrieves the image properties for each file. This approach provides poor performance.

C#